L-Università ta' Malta
Faculty of Information & Communication Technology
Department of Computer Information Systems

Create An App: Qontract

Nico Ismaili* (2111912)
*VISITING

Study-unit: **Mobile Computing**
Code: **CIS2208**
Lecturer: **Dr Conrad Attard**

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as “the unacknowledged use, as one's own, of work of another person, whether or not such work has been published, and as may be further elaborated in Faculty or University guidelines" (University Assessment Regulations, 2009, Regulation 39 (b)(i), University of Malta).

I / We*, the undersigned, declare that the assignment submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for committing a breach of the regulations include loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected and will be given zero marks.

* Delete as appropriate.

(N. B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

Nico Ismaili

Student Name | Signature

CIS2208 | Qontract

Course Code | Title of work submitted

20.05.2022

Date

Contents

1. Concept

The publication and selling of images containing people legally requires a model release form, it enables a photographer to publish and sell images of the depicted person. To facilitate the process of creating, managing, and signing such contracts Nico Ismaili has proposed an application, in which the user (i.e., the photographer) can generate said forms. These will then be prefilled with the user's data and made accessible in a searchable and filterable list. Each contract contains metadata which can be shared via a QR-code.

2. Project Management

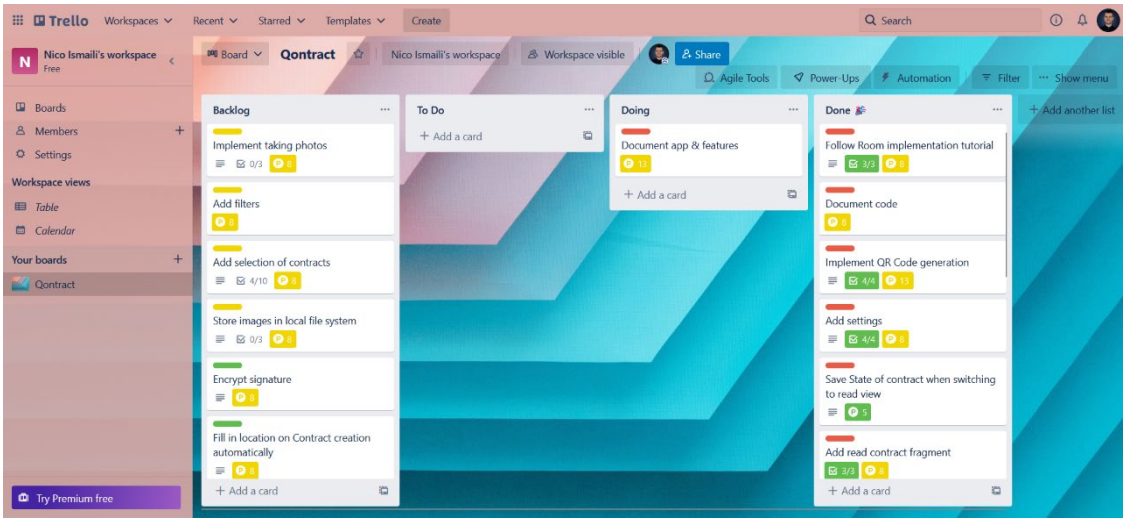

Figure 2.2.1 Screenshot of the Trello Board

This project was managed with the help of Trello. A Kanban Board was used to enable agile development and progress tracking.

3. Components

1.1 Main Screen / Activity

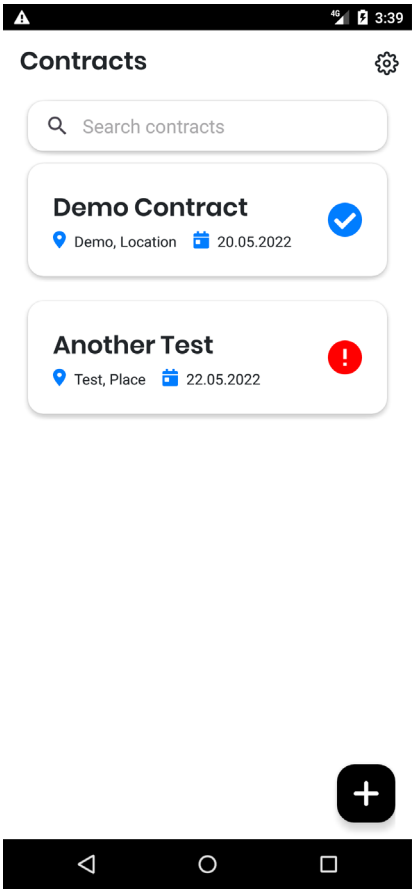

Figure 3.1 Screenshot of the main screen

1.1.1 Features

The main screen of the app presents a searchable overview of all contracts saved on the device. From it, the user can either open details of a certain contract, add a new contract or navigate to the settings screen. A contract item contains the most vital information about the contract such as the place and date it was created as well as the name of the model. The icon on the right side of the contract indicates if all required fields have been filled out or not. If the icon is a blue checkmark, everything has been filled out, otherwise the app will display a red exclamation mark instead.

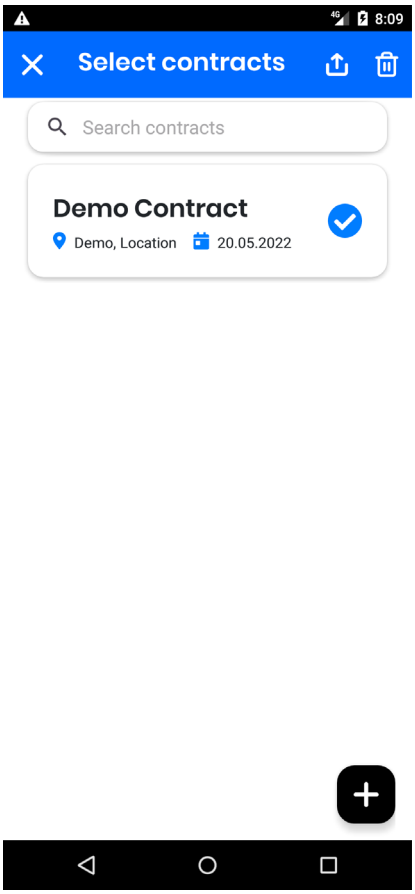

Figure 3.2 Screenshot of the ActionMode

The user can also enter an ActionMode by long clicking on any of the displayed contracts. This gives him or her the option to share or delete a group of contracts by selecting them. This feature is currently still in alpha and needs improvement.

1.1.2 Composition

To ensure the app is usable on a multitude of devices it only has one main Activity and a plethora of fragments that contain each feature. Said activity contains the AppBar which presents a different menu (and title) depending on where in the app the user currently is. In this case, the AppBar allows the user to navigate to the settings fragment.
Below the AppBar the most vital component of the application resides: The NavHostFragment, which is a key component to enable dynamic navigation of the app. In short, the component is a container which can switch fragments in and out. It is a core component of Android's Navigation API. The Navigation API handles automatic AppBar updates, manages the back stack, fragment relationships via the Navigation Graph (see figure 3.3) and animations between screen transitions.

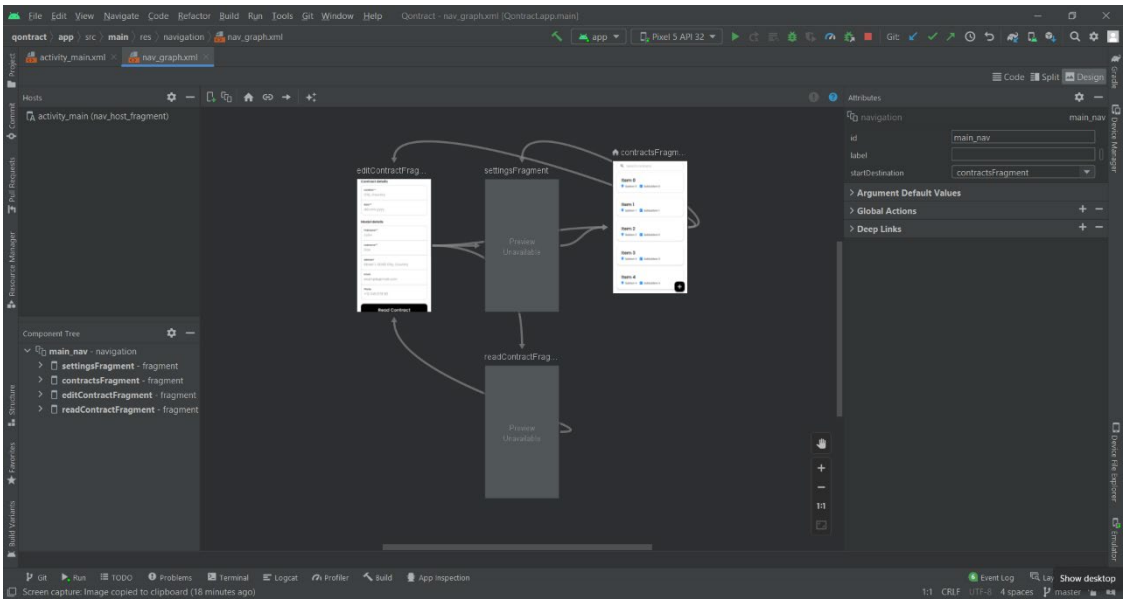

Figure 3.3 Screenshot of the navigation graph

1.1.2.1 Contracts Fragment

Each fragment's composition is determined by a corresponding layout. In the main screen the NavHostFragment displays the contract fragment: An overview screen showing all contracts. It consists of the following components.

Below the AppBar a SearchView displays a search bar. It provides an interface with which the user can search for contracts by a model's name. Entering any character into the search field will automatically update the list of contracts below the search bar.

The list of contracts is managed by a RecyclerView which is bound to a LiveData object. This ensures that the exact state of all contracts currently in the database is always displayed correctly. Each item or contract in the displayed list is clickable and will open the edit contract fragment when clicked. As stated above, a long click on any of the contracts will activate an ActionMode provides additional functionality.

Before any contracts can be clicked they first need to be added, this can be done by pressing on the FloatingActionButton in the bottom right corner. This will forward the user to the add contract fragment.

1.2 Edit / Add Contract Fragment

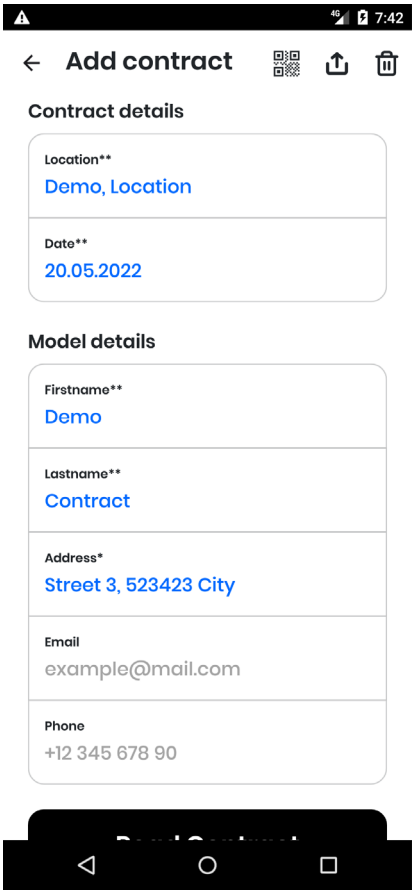

Figure 3.4 Screenshot of upper half of edit contract screen

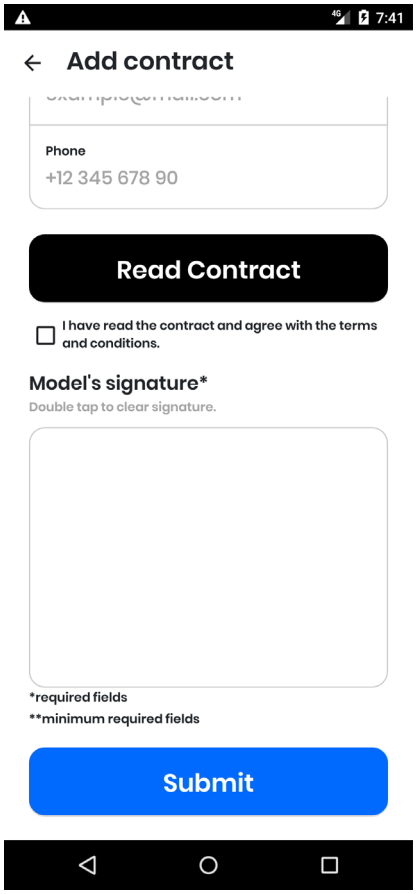

Figure 3.5 Screenshot of lower half of edit contract screen

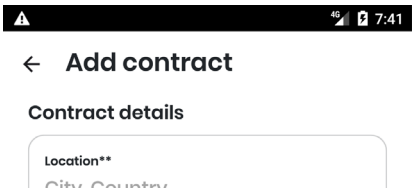

Figure 3.6 Screenshot of menu in add contract / model mode

1.2.1 Features

The edit / add contract fragment is the centrepiece of the entire application. It provides all the necessary functionality to edit and manage contracts. Here the user can input a model's details, navigate to the read contract fragment and back, accept the contracts terms as well as sign and submit the contract.

The user can also generate a QR code of a contract, delete and share a contract. Although, the menu items to use said features are only visible if the model mode is inactive and the current contract is not a new contract (compare figures 3.5 and 3.6).
To prevent the user from accidentally deleting contracts or submitting unfinished contracts several dialogs have been implemented to aid the user.

1.2.1.1 Delete Prompt

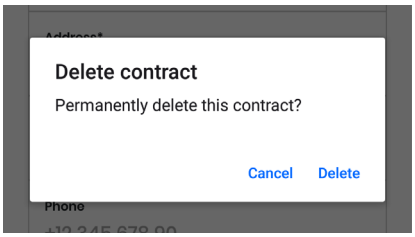

Figure 3.7 Screenshot of delete contract dialog

Figure 3.7 shows a simple confirmation prompt which is displayed when a user presses the delete button. The user hast the option to cancel or proceed with the deletion of a contract.

1.2.1.2 Submission Errors

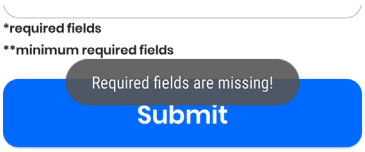

Figure 3.8 Screenshot of "required fields missing" error dialog

When editing a contract and model mode is activated, the user will not be able to submit a form without filling out every required field (marked with a *). This is done to ensure a contract is not submitted in an unfinished state.

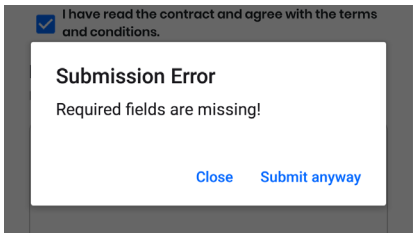

Figure 3.9 Screenshot of "required fields missing" Toast

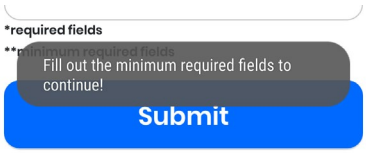

Figure 3.10 Screenshot of "minimum required field missing" Toast

On the other hand, if model mode is deactivated, the user should have a lot more freedom when editing a contract. Namely because one can assume that if model mode is inactive, the photographer is currently operating the device. Contracts should therefore be submittable in an unfinished state to allow for example, the creation of contracts before a photoshoot. As can be seen in figure 3.9, if a contract is submitted with missing required fields, the user has the option to submit it anyways, although this will result in the contract being displayed with a red exclamation mark icon, to indicate its unfinished state, in the overview. There are however some fields which at minimum are required (marked by **) to be able to submit a contract. These fields include the first and last name of a model and the location and date of the contract. If said fields are not filled out at the point of submission the user will receive the Toast message depicted in figure 3.10.

1.2.1.3 QR Code generation

Figure 3.11 Screenshot of QR code dialog

As soon as the user clicks the QR-code button whilst in the edit contract fragment, a GET-request is sent to a QR-Code REST API containing the most vital information of the currently open contract. The response of the request is an image of a working QR-code (see figure

3.12) which will be displayed in a dialog window where the user can scan the QR-code to retrieve a contract's data. The user then has the option to close the dialog after he or she is done using the QR-code.

1.2.1.4 Share / Export Screen

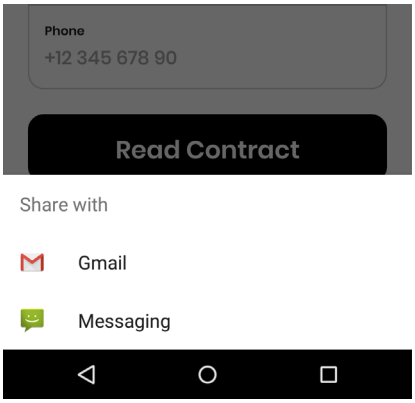

Figure 3.12 Screenshot of share dialog

1.2.1.5 Share / Export Error

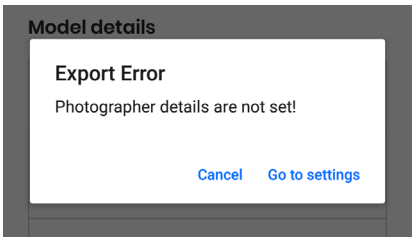

Figure 3.13 Screenshot of export error dialog

When either trying to export a contract via sharing or QR-code, a check is preformed to ensure the photographer details have been set so they can be included in the exported contract. If they have not been set, the user is prompted with a dialog reminding him of the previously explained necessity (see figure 3.11). The user then has the option to directly navigate to the settings fragment from the dialog or cancel the exporting process.

1.2.2 Composition

The bulk of the edit / add contract fragment is made up of TextViews for each necessary input field. In addition to this, two Buttons were used for the submission of the contract and accessing the actual contract text. To enable the user to agree to the terms of the contract a

simple Switch was used. Finally, a SignaturePad was used to implement the signing of the contract.

The QR-Code is contained inside an ImageView which itself is contained inside of a Dialog. The actual QR-code image itself is gotten via a HTTP-request provided by Glide, a library which facilitates the accessing and processing of images and is recommended by Google. Sharing is achieved by using an Intent and giving it the ACTION_SEND action, which in turn opens the default share view contained in the Android OS.

Lastly, the differentiation between a new and a pre-existing contract is done by inserting a Boolean into the Bundle that is received by the edit / add fragment every time it is opened. This guarantees the right AppBar no matter from where the user accesses this fragment.

1.3 Settings Fragment

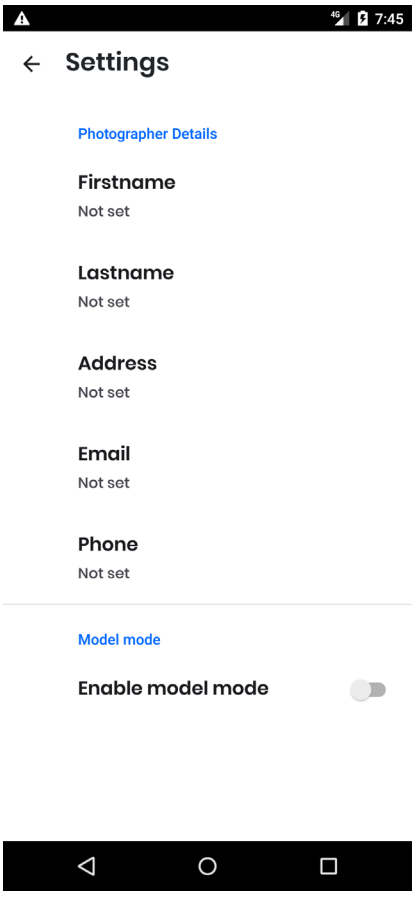

Figure 3.14 Screenshot of the settings screen

1.3.1 Features

The settings fragment only has two main purposes, the first of which is giving the photographer the ability to input his personal details which are necessary since he or she is

the first party in every contract managed by the app. The second purpose of the settings screen is to toggle model mode. Model mode disables the possibility to submit contracts before all required fields are set and hides the delete, share and QR code menu items when editing or creating a contract.

1.3.2 Composition

When the settings fragment is being shown the AppBar only enables the user to navigate back to the contracts fragment and does not provide any additional menu items.
The fragment itself is a subclass of the PreferenceFragmentCompat Class. It inflates a PreferenceScreen containing an EditTextPreferences for each personal detail of the photographer and one SwitchPreference to toggle model mode. Preferences are stored on the device and persist even after closing the app. They can be accessed from any Fragment by instantiating a SharedPreferences object, therefore providing a very robust solution for saving and accessing user settings.

1.4 Read Contract Screen

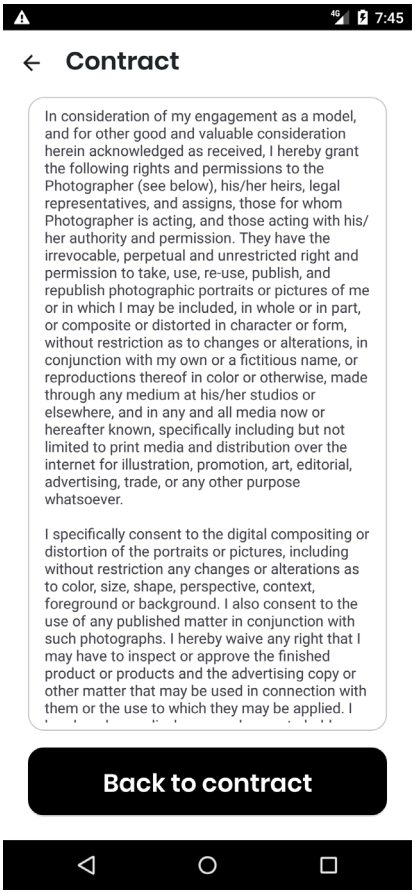

Figure 3.15 Screenshot of read contract screen

1.4.1 Features

The read contract Fragment is the simplest fragment in the app. Its only function is to display the model release form text and enable the user to navigate back to editing the app when he or she is done reading.

1.4.2 Composition

The fragment is made up of two components the first one being a ScrollView which contains a TextView displaying the contract text. The second component is a Button with which the user can navigate back to the edit contract fragment.

4. Design Choices

Fundamentally this application is based on Material Design 3. In general Material Design is a tried and tested set of rules and guides to creating user experience friendly Android applications.

Although the most vital design decisions, such as the correct margins, padding, components and so on, oblige the Material principles in a one-to-one manner, some alterations were performed to distinguish the design of the application in a direct juxtaposition of itself and other applications, that are based on Material Design.

The first of said decisions is the colours used throughout the app. Material Design recommends implementing dynamic colours, which adjust to the user's theme preferences, but, as this application follows a certain branding, its design opts for a more static presentation. Nonetheless, the app does contain the recommended number of colours, namely a primary, secondary, and tertiary colour to represent the possible actions, a neutral key colour and a variant of the neutral key colour.

In correlation to the usage of certain colours another choice was made, which was the change of font. Instead of using the default Android font Roboto, the Google Font "Poppins" was used to further support branding. To provide the user with a more friendly appearance of the app mostly medium to large corner radii were used.

A bigger deviation from Material Design can be seen when looking at the search bar in the contracts fragment and the form fields in the edit / add contract fragment.

The search bar was modified to tie in the design of the contract ViewHolders together with itself. To achieve this the SearchView's border was rounded, the view itself was elevated and the underline was removed.

The form fields have a fair amount of resemblance with the text fields introduced in Material Design 3 but provide a larger label text size to combat the legibility issue that occurred with use of the primary colour as the text colour. Furthermore, making the border a custom one facilitated the grouping of related fields with each other, which again aids the intuitiveness of the app.

5. Next Steps

The here presented application is a working beta of what the final application will be, therefore there are many features which could be introduced in the coming versions of this application.
The first major improvement that can be made is the introduction of a proprietary REST API to handle processes such as the generation of QR-codes, converting the exported contract data into a PDF and storing the inputted contracts in a cloud-based storage. This could then even be expanded into a cross platform contract management software.
After this, the multi selection feature of the app needs improvement. As stated in chapter 1.1, this feature is still in an alpha state and needs improvement to better the intuitiveness of its use. The creation of the aforementioned API would greatly facilitate the implementation of this feature.
Although the previously described features are the two major contenders that would appear in an actual release of the app, there are still several noteworthy features and improvements that could be introduced in future versions. Such as:

- Encrypting the Base64 String containing the signature before saving it
- Caching the QR-code image to prevent unnecessary internet usage
- Using fingerprint or pin authentication to prevent the unauthorized changing of the settings
- Filling in the contract location using GPS data
- Adding search filters
- Adding the ability to attach images to a contract
- Adding data binding to the contract ViewHolder

6. Appendix

1.5 Bibliography

Throughout the making of this application a variety of sources were used as reference or guidance. The links to these sources are listed here:

https://www.geeksforgeeks.org/searchview-in-android-with-recyclerview/
https://www.youtube.com/watch?v=6qYNxtG0LNE
https://developer.android.com/guide/topics/search

https://www.youtube.com/watch?v=M73Vec1oieM

https://www.geeksforgeeks.org/searchview-in-android-with-recyclerview/

https://stackoverflow.com/a/59743519/13929165

https://stackoverflow.com/a/58352437/13929165

https://medium.com/@sienatime/enabling-sqlite-fts-in-room-2-1-75e17d0f0ff8

https://www.youtube.com/watch?v=CTvzoVtKoJ8

https://developer.android.com/topic/libraries/data-binding

https://developer.android.com/guide/topics/graphics/vector-drawable-resources

https://github.com/gcacace/android-signaturepad

https://developer.android.com/training/data-storage/room/referencing-data

https://developer.android.com/guide/fragments/saving-state

https://developer.android.com/guide/topics/ui/settings

https://www.geeksforgeeks.org/how-to-generate-qr-code-in-android/

https://goqr.me/api/

https://stackoverflow.com/a/61290943/13929165

https://developer.android.com/codelabs/android-room-with-a-view#13

https://developer.android.com/training/data-storage/room/defining-data.html